5-verify_audit_batch.py

User Instructions

Overview

`5-verify_audit_batch.py` is used to compare a publicly anchored manufacturing batch against a local audit bundle.

The program performs: - cryptographic verification - Merkle proof validation - audit bundle comparison - manufacturing record consistency checks

The verifier is intended to demonstrate: - manufacturing accountability - tamper-evidence concepts - audit consistency - decentralized verification workflows

This script is primarily intended for: - local auditing - standalone verification - offline demonstrations - direct review of audit archives

The separate:

`Audit Portal Dashboard`

extends this concept into a remote multi-user auditing system.

Launching the Program

Run the script from a terminal:

```
python 5-verify_audit_batch.py
```

A Tkinter graphical interface window will appear.

Purpose of Verification

The manufacturing workflow creates several layers of accountability:

1. Manufacturing job records
2. Merkle batches
3. Public anchors
4. Audit bundles

The public batch demonstrates: - that a manufacturing batch existed - when it existed - the cryptographic structure of the batch

The audit bundle preserves: - the detailed local records - manufacturing manifests - append-only logs - optional G-code

The verifier compares these two sources to determine whether: - the local records are consistent - files appear missing - manufacturing history was modified - Merkle proofs remain valid

---

Interface Sections

The interface contains sections used to select verification files and review results.

Public Batch Selection

Use the:

`Browse`

button to select the public manufacturing batch JSON file.

Example:

`batch_ForgeWorks_Printer01_9f2a1c4d.json`

This file is typically generated by:

`2-merkle_batch_generator.py`

and may later be publicly anchored using:

`3-merkle_github_anchor.py`

The public batch contains: - Merkle root - leaf hashes - proofs - job references - batch metadata

The verifier uses this file as the public accountability reference.

Local Audit Bundle Selection

Use the:

`Browse`

button to select the local audit bundle.

The selected input may be: - an extracted audit folder - an audit ZIP archive

The audit bundle is typically generated by:

`4-build_audit_bundle_gui.py`

Example:

`audit_bundle_ForgeWorks_Printer01.zip`

or:

`audit/audit_20260524_153045/`

The audit bundle contains the detailed manufacturing records used for local comparison.

Verify Button

Press:

`Verify`

to begin the verification process.

When pressed, the program:

1. Reads the public batch JSON
2. Validates Merkle proofs
3. Recomputes Merkle roots
4. Reads the local audit bundle
5. Locates manufacturing manifests
6. Recomputes local leaf hashes
7. Compares local records to the public batch
8. Generates a verification report

Verification Report

After verification completes, the:

`Verification Report`

window displays a human-readable summary of the audit results.

The report is divided into sections.

Public Verification

This section validates the public batch itself.

It may include: - public batch path - PASS or FAIL status - Merkle root validation - proof validation - batch hash validation - batch structure findings - public batch warnings

This section confirms whether the public manufacturing batch appears internally consistent.

Local Bundle Comparison

This section compares the audit bundle against the public batch.

It may include: - audit bundle location - PASS or FAIL status - matched jobs - missing jobs - extra local jobs - leaf hash mismatches - registry findings - append-log findings

This section determines whether the local manufacturing records remain consistent with the publicly referenced batch.

Overall Result

The final section displays the overall verification result.

Example:

```
Overall Status: PASS
```

or:

```
Overall Status: FAIL
```

A PASS result indicates: - the public batch structure validated - the local audit bundle matched the public references

A FAIL result indicates: - inconsistencies - missing records - modified records - invalid proofs - or structural mismatches

---

Meaning of Verification

The verifier does NOT determine: - legality - intent - safety - policy compliance

Instead, it demonstrates: - whether records appear internally consistent - whether manufacturing history appears modified - whether audit references remain cryptographically linked

The system focuses on: - accountability - continuity - tamper evidence - voluntary transparency

rather than restrictive prevention systems.

Finding Verification Files

Public Batch Files

Public batch JSON files are typically located near:

`output/<facility_id>/<instrument_id>/jobs/batches/`

Examples:

`output/ForgeWorks/Printer01/jobs/batches/`

Example filename:

`batch_ForgeWorks_Printer01_9f2a1c4d.json`

Audit Bundles

Audit bundles are typically located in:

`audit/audit_<timestamp>/`

Example:

`output/ForgeWorks/Printer01/jobs/audit/audit_20260524_153045/`

Inside this folder:

`audit_bundle_ForgeWorks_Printer01.zip`

contains the portable audit package.

Local Verification vs Remote Verification

This verifier script is primarily intended for: - local auditing - direct review - standalone demonstrations - offline verification workflows

The separate:

`Audit Portal Dashboard`

extends these ideas into: - remote auditing - multi-user verification - manufacturer/auditor roles - uploaded audit workflows - centralized report management

Both systems use the same underlying accountability concepts.

Recommended Workflow

Typical usage:

1. Generate manufacturing jobs using:

   `1-merkle_integrated_slicer.py`

2. Generate a manufacturing batch using:

   `2-merkle_batch_generator.py`

3. Optionally publicly anchor the batch using:

   `3-merkle_github_anchor.py`

4. Build an audit bundle using:

   `4-build_audit_bundle_gui.py`

5. Open:

   `5-verify_audit_batch.py`

6. Select:

   – the public batch JSON
   – the local audit bundle

7. Press:

   `Verify`

8. Review the Verification Report

Purpose of the Demonstration

This verifier demonstrates: - local manufacturing verification - cryptographic audit comparison - Merkle proof validation - tamper-evidence concepts - decentralized accountability workflows

The goal of the demonstration is to encourage exploration of: - voluntary manufacturing accountability - decentralized audit systems - cryptographic manufacturing continuity - collaborative trust frameworks